

UNIVERSIDAD ESPÍRITU SANTO

Proyecto Integrador - Diseño de Software
FASE 2: Código base e informe de diagnóstico

Sistema de Reservas de Canchas Deportivas

Equipo: _____

Integrantes: _____

Fecha: _____

1. Objetivo del entregable

Presentar una implementación funcional mínima de tres casos de uso del sistema de reservas, una suite inicial de pruebas unitarias ejecutándose correctamente y un diagnóstico documentado de los malos olores presentes en el código base antes de aplicar las refactorizaciones de la Fase 3.

2. Casos de uso implementados

- CU-01 Crear reserva: valida datos, disponibilidad, registra la reserva y genera una notificación.
- CU-02 Consultar disponibilidad: determina si una cancha está libre para un período solicitado.
- CU-03 Cancelar reserva: localiza una reserva, actualiza su estado y genera una notificación.

3. Organización del código

El proyecto utiliza Python con orientación a objetos y separa entidades, repositorio, notificaciones, servicios y pruebas. La persistencia es en memoria para mantener el alcance controlado y permitir pruebas reproducibles sin infraestructura externa.

Ubicación	Responsabilidad
src/reservas/models.py	Entidades Client, Court y Reservation.
src/reservas/repository.py	Repositorio en memoria y detección de cruces de horarios.
src/reservas/service.py	Implementación de los tres casos de uso.
src/reservas/notification.py	Simulación del envío de notificaciones.
tests/test_reservation_service.py	Suite inicial de pruebas unitarias.

4. Resultado de pruebas unitarias

La suite fue ejecutada con pytest. Resultado obtenido: 7 pruebas aprobadas.

```
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.0.2, pluggy-1.6.0 -- /opt/pyvenv/bin/python
cachedir: .pytest_cache
metadata: {'Python': '3.13.5', 'Platform': 'Linux-6.12.13-x86_64-with-glibc2.41', 'Packages': {'pytest': '9.0.2', 'pluggy': '1.6.0'}, 'Plugins':
{'cov': '7.0.0', 'json-report': '1.5.0', 'anyio': '4.13.0', 'Faker': '40.1.2', 'ddtrace': '4.4.0', 'metadata': '3.1.1', 'asyncio': '1.3.0'}, 'PLATFORM':
'linux/amd64', 'CI': 'true'}
rootdir: /mnt/data/Fase_2_Sistema_Reservas_Canchas
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.0.0, json-report-1.5.0, anyio-4.13.0, Faker-40.1.2, ddtrace-4.4.0, metadata-3.1.1, asyncio-1.3.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collecting ... collected 7 items

tests/test_reservation_service.py::test_create_reservation_successfully PASSED [ 14%]
tests/test_reservation_service.py::test_rejects_conflicting_reservation PASSED [ 28%]
tests/test_reservation_service.py::test_check_availability_returns_true_for_free_slot PASSED [ 42%]
tests/test_reservation_service.py::test_check_availability_returns_false_for_occupied_slot PASSED [ 57%]
tests/test_reservation_service.py::test_cancel_existing_reservation PASSED [ 71%]
tests/test_reservation_service.py::test_cannot_cancel_unknown_reservation PASSED [ 85%]
tests/test_reservation_service.py::test_rejects_invalid_email PASSED [100%]

===== 7 passed in 0.03s =====
```

5. Catálogo de malos olores

5.1 Método largo

Ubicación: src/reservas/service.py, líneas 17-83

Justificación: El método `create_reservation` concentra validación de datos, comprobación de disponibilidad, creación de entidades, persistencia y notificación. Esto dificulta la lectura, eleva la complejidad y obliga a modificar el mismo método cuando cambia cualquiera de esas responsabilidades.

Refactorización prevista: Extract Method y separación de validadores, fábrica de reservas y servicio de notificación.

Mal olor 1 - Método largo: create_reservation

```
17 |
18 | def create_reservation(
19 |     self,
20 |     client_identification: str,
21 |     client_name: str,
22 |     client_email: str,
23 |     court_id: str,
24 |     court_name: str,
25 |     sport: str,
26 |     start_at: datetime,
27 |     duration_hours: int,
28 |     notes: str = "",
29 | ) -> Reservation:
30 |     # Versión base intencionalmente mejorable para la fase de diagnóstico.
31 |     if client_identification is None or client_identification.strip() == "":
32 |         raise ValueError("La identificación del cliente es obligatoria")
33 |     if client_name is None or client_name.strip() == "":
34 |         raise ValueError("El nombre del cliente es obligatorio")
35 |     if client_email is None or client_email.strip() == "":
36 |         raise ValueError("El correo del cliente es obligatorio")
37 |     if "@" not in client_email or "." not in client_email:
38 |         raise ValueError("El correo del cliente no es válido")
39 |     if court_id is None or court_id.strip() == "":
40 |         raise ValueError("El identificador de la cancha es obligatorio")
41 |     if court_name is None or court_name.strip() == "":
42 |         raise ValueError("El nombre de la cancha es obligatorio")
43 |     if sport is None or sport.strip() == "":
44 |         raise ValueError("El deporte es obligatorio")
45 |     if start_at is None:
46 |         raise ValueError("La fecha y hora son obligatorias")
47 |     if start_at <= datetime.now():
48 |         raise ValueError("La reserva debe programarse para una fecha futura")
49 |     if duration_hours < 1 or duration_hours > 4:
50 |         raise ValueError("La duración debe estar entre 1 y 4 horas")
51 |     if len(notes) > 250:
52 |         raise ValueError("Las observaciones no pueden superar 250 caracteres")
53 |
54 |     conflicts = self.repository.find_active_by_court_and_period(
55 |         court_id, start_at, duration_hours
56 |     )
57 |     if len(conflicts) > 0:
58 |         raise ValueError("La cancha no está disponible en el horario solicitado")
59 |
60 |     client = Client(client_identification, client_name, client_email)
61 |     court = Court(court_id, court_name, sport, True)
62 |     reservation = Reservation(
63 |         reservation_id=str(uuid4()),
64 |         client=client,
65 |         court=court,
66 |         start_at=start_at,
67 |         duration_hours=duration_hours,
68 |         status="CONFIRMED",
69 |         notes=notes,
70 |     )
71 |     self.repository.save(reservation)
72 |     self.notification_service.send(
73 |         client.email,
74 |         "Reserva confirmada para "
75 |         + court.name
76 |         + " el "
77 |         + start_at.strftime("%Y-%m-%d %H:%M")
78 |         + " por "
79 |         + str(duration_hours)
80 |         + " hora(s).",
81 |     )
82 |     return reservation
83 |
```

5.2 Lista larga de parámetros

Ubicación: src/reservas/service.py, líneas 17-30

Justificación: El caso de uso recibe nueve datos individuales. La firma es difícil de recordar, permite intercambiar argumentos del mismo tipo y crecerá cuando se agreguen nuevos datos.

Refactorización prevista: Introduce Parameter Object mediante una clase ReservationRequest.

Mal olor 2 - Lista larga de parámetros

```
17 |  
18 |     def create_reservation(  
19 |         self,  
20 |         client_identification: str,  
21 |         client_name: str,  
22 |         client_email: str,  
23 |         court_id: str,  
24 |         court_name: str,  
25 |         sport: str,  
26 |         start_at: datetime,  
27 |         duration_hours: int,  
28 |         notes: str = "",  
29 |     ) -> Reservation:  
30 |         # Versión base intencionalmente mejorable para la fase de diagnóstico.
```

5.3 Código duplicado

Ubicación: src/reservas/service.py, líneas 85-101 y 31-53

Justificación: Las validaciones de court_id, start_at y duration_hours aparecen tanto en create_reservation como en check_availability. La duplicación incrementa el mantenimiento y puede producir reglas inconsistentes.

Refactorización prevista: Extract Method para centralizar la validación del período solicitado.

Mal olor 3 - Validaciones duplicadas

```
85 |         self, court_id: str, start_at: datetime, duration_hours: int  
86 |     ) -> bool:  
87 |         if court_id is None or court_id.strip() == "":  
88 |             raise ValueError("El identificador de la cancha es obligatorio")  
89 |         if start_at is None:  
90 |             raise ValueError("La fecha y hora son obligatorias")  
91 |         if duration_hours < 1 or duration_hours > 4:  
92 |             raise ValueError("La duración debe estar entre 1 y 4 horas")  
93 |         conflicts = self.repository.find_active_by_court_and_period(  
94 |             court_id, start_at, duration_hours  
95 |         )  
96 |         return len(conflicts) == 0  
97 |  
98 |     def cancel_reservation(self, reservation_id: str, reason: str) -> Reservation:  
99 |         if reservation_id is None or reservation_id.strip() == "":  
100 |             raise ValueError("El identificador de la reserva es obligatorio")  
101 |         if reason is None or reason.strip() == "":
```

5.4 Obsesión por tipos primitivos

Ubicación: src/reservas/models.py, líneas 18-27

Justificación: El estado de la reserva se representa con string y la duración con un entero sin tipos de dominio. Esto permite valores inválidos como CONFIRMD o duraciones fuera de rango.

Refactorización prevista: Replace Type Code with Enum y creación de objetos de valor para duración e identificadores.

Mal olor 4 - Obsesión por tipos primitivos

```
18 |  
19 |  
20 | @dataclass  
21 | class Reservation:  
22 |     reservation_id: str  
23 |     client: Client  
24 |     court: Court  
25 |     start_at: datetime  
26 |     duration_hours: int  
27 |     status: str
```

5.5 Números mágicos

Ubicación: src/reservas/service.py, líneas 48-53

Justificación: Los límites 1, 4 y 250 aparecen directamente en las condiciones. Su significado no está expresado mediante nombres y cualquier cambio obliga a localizar valores dispersos.

Refactorización prevista: Replace Magic Number with Symbolic Constant o política de reservas configurable.

Mal olor 5 - Números mágicos

```
48 |         raise ValueError("La reserva debe programarse para una fecha futura")  
49 |         if duration_hours < 1 or duration_hours > 4:  
50 |             raise ValueError("La duración debe estar entre 1 y 4 horas")  
51 |         if len(notes) > 250:  
52 |             raise ValueError("Las observaciones no pueden superar 250 caracteres")  
53 |
```

5.6 Clase con múltiples responsabilidades

Ubicación: src/reservas/service.py, líneas 8-119

Justificación: ReservationService valida datos, construye entidades, consulta disponibilidad, persiste, cambia estados y compone mensajes. Esta concentración reduce la cohesión y complica las pruebas aisladas.

Refactorización prevista: Extract Class para ReservationValidator, ReservationFactory y ReservationNotifier.

Mal olor 6 - Clase con múltiples responsabilidades

```
8 |
9 | class ReservationService:
10 |     def __init__(
11 |         self,
12 |         repository: InMemoryReservationRepository,
13 |         notification_service: NotificationService,
14 |     ) -> None:
15 |         self.repository = repository
16 |         self.notification_service = notification_service
17 |
18 |     def create_reservation(
19 |         self,
20 |         client_identification: str,
21 |         client_name: str,
22 |         client_email: str,
23 |         court_id: str,
24 |         court_name: str,
25 |         sport: str,
26 |         start_at: datetime,
27 |         duration_hours: int,
28 |         notes: str = "",
29 |     ) -> Reservation:
30 |         # Versión base intencionalmente mejorable para la fase de diagnóstico.
31 |         if client_identification is None or client_identification.strip() == "":
32 |             raise ValueError("La identificación del cliente es obligatoria")
33 |         if client_name is None or client_name.strip() == "":
34 |             raise ValueError("El nombre del cliente es obligatorio")
35 |         if client_email is None or client_email.strip() == "":
36 |             raise ValueError("El correo del cliente es obligatorio")
37 |         if "@" not in client_email or "." not in client_email:
38 |             raise ValueError("El correo del cliente no es válido")
39 |         if court_id is None or court_id.strip() == "":
40 |             raise ValueError("El identificador de la cancha es obligatorio")
41 |         if court_name is None or court_name.strip() == "":
42 |             raise ValueError("El nombre de la cancha es obligatorio")
43 |         if sport is None or sport.strip() == "":
44 |             raise ValueError("El deporte es obligatorio")
45 |         if start_at is None:
46 |             raise ValueError("La fecha y hora son obligatorias")
47 |         if start_at <= datetime.now():
48 |             raise ValueError("La reserva debe programarse para una fecha futura")
49 |         if duration_hours < 1 or duration_hours > 4:
50 |             raise ValueError("La duración debe estar entre 1 y 4 horas")
51 |         if len(notes) > 250:
52 |             raise ValueError("Las observaciones no pueden superar 250 caracteres")
53 |
54 |         conflicts = self.repository.find_active_by_court_and_period(
55 |             court_id, start_at, duration_hours
56 |         )
57 |         if len(conflicts) > 0:
58 |             raise ValueError("La cancha no está disponible en el horario solicitado")
59 |
60 |         client = Client(client_identification, client_name, client_email)
61 |         court = Court(court_id, court_name, sport, True)
62 |         reservation = Reservation(
63 |             reservation_id=str(uuid4()),
64 |             client=client,
65 |             court=court,
66 |             start_at=start_at,
67 |             duration_hours=duration_hours,
68 |             status="CONFIRMED",
69 |             notes=notes,
70 |         )
71 |         self.repository.save(reservation)
72 |         self.notification_service.send(
73 |             client.email,
74 |             "Reserva confirmada para "
75 |             + court.name
76 |             + " el "
```

6. Conclusiones

La implementación base cumple los tres casos de uso y mantiene una suite de pruebas en verde. Los malos olores identificados no impiden el funcionamiento actual, pero evidencian riesgos de mantenibilidad, extensibilidad y consistencia. En la Fase 3 se aplicarán refactorizaciones pequeñas, cada una respaldada por la ejecución de las pruebas y un commit independiente.

7. Evidencias incluidas en el repositorio

- Código fuente funcional.
- Siete pruebas unitarias.
- Archivo resultado_pruebas.txt con la ejecución de pytest.
- Capturas de los seis malos olores.
- Informe editable en Word y versión final en PDF.
- README con instrucciones de ejecución y publicación en GitHub/GitLab.